

ZK Battleship

6.5610 Project

Timothy Mathew* Varun Sangtani*,[†]
Zachary S. Siegel* Aleksander T. Vestlund*,[‡]

*EECS, MIT

[†]EEE, ICL

[‡]IIK, NTNU

May 4, 2026

Outline

- 1 Motivation
- 2 Demo
- 3 Overview of Zero-Knowledge Proofs
- 4 Battleship Circuits
- 5 Merkle Trees

Outline

- 1 Motivation
- 2 Demo
- 3 Overview of Zero-Knowledge Proofs
- 4 Battleship Circuits
- 5 Merkle Trees

The Trust Gap in Digital Games

- Traditional Battleship requires a **hidden state** where players must keep ship locations secret while proving they are not cheating.

The Trust Gap in Digital Games

- Traditional Battleship requires a **hidden state** where players must keep ship locations secret while proving they are not cheating.
- Games rely on a **central server** to verify board validity and hits.

The Trust Gap in Digital Games

- Traditional Battleship requires a **hidden state** where players must keep ship locations secret while proving they are not cheating.
- Games rely on a **central server** to verify board validity and hits.
- If the server is compromised or biased, the game's integrity is lost.

The Trust Gap in Digital Games

- Traditional Battleship requires a **hidden state** where players must keep ship locations secret while proving they are not cheating.
- Games rely on a **central server** to verify board validity and hits.
- If the server is compromised or biased, the game's integrity is lost.
- In a peer-to-peer (P2P) environment, how do you prove a **miss** without revealing where your ships actually are?

Why Zero-Knowledge Proofs?

The Goal

Prove the validity of every move without revealing the underlying board configuration.

Why Zero-Knowledge Proofs?

The Goal

Prove the validity of every move without revealing the underlying board configuration.

Integrity Ensure the opponent did not move their ships mid-game, and hits are reported correctly.

Why Zero-Knowledge Proofs?

The Goal

Prove the validity of every move without revealing the underlying board configuration.

- Integrity** Ensure the opponent did not move their ships mid-game, and hits are reported correctly.
- Privacy** Keep (unreported) ship coordinates (x, y) private, even after the game ends.

Why Zero-Knowledge Proofs?

The Goal

Prove the validity of every move without revealing the underlying board configuration.

Integrity Ensure the opponent did not move their ships mid-game, and hits are reported correctly.

Privacy Keep (unreported) ship coordinates (x, y) private, even after the game ends.

Verifiability Anyone can verify the proof π for a hit or miss.

Demo

Outline

- 1 Motivation
- 2 Demo
- 3 Overview of Zero-Knowledge Proofs**
- 4 Battleship Circuits
- 5 Merkle Trees

Zero-Knowledge Proofs

Completeness If a statement is **true**, an honest prover can convince an honest verifier.

Zero-Knowledge Proofs

Completeness If a statement is **true**, an honest prover can convince an honest verifier.

Soundness If a statement is **false**, a cheating prover cannot convince a verifier.

Zero-Knowledge Proofs

Completeness If a statement is **true**, an honest prover can convince an honest verifier.

Soundness If a statement is **false**, a cheating prover cannot convince a verifier.

Zero-Knowledge The verifier learns nothing except the validity of the statement.

Zero-Knowledge Proofs

Completeness If a statement is **true**, an honest prover can convince an honest verifier.

Soundness If a statement is **false**, a cheating prover cannot convince a verifier.

Zero-Knowledge The verifier learns nothing except the validity of the statement.

Examples from class: 3Col and Sumcheck.

- An improvement upon the zk-Succinct Non-interactive Argument of Knowledge (SNARK) Pinocchio discussed in class.

- An improvement upon the zk-Succinct Non-interactive Argument of Knowledge (SNARK) Pinocchio discussed in class.
- **Small proofs:** Constant size regardless of circuit complexity.

- An improvement upon the zk-Succinct Non-interactive Argument of Knowledge (SNARK) Pinocchio discussed in class.
- **Small proofs:** Constant size regardless of circuit complexity.
- Only 3 pairings required to verify any proof.

- An improvement upon the zk-Succinct Non-interactive Argument of Knowledge (SNARK) Pinocchio discussed in class.
- **Small proofs:** Constant size regardless of circuit complexity.
- Only 3 pairings required to verify any proof.
- Therefore, both fast proving and verification times.

Outline

- 1 Motivation
- 2 Demo
- 3 Overview of Zero-Knowledge Proofs
- 4 Battleship Circuits**
- 5 Merkle Trees

Circom: A Language for Zero-Knowledge Circuits

- **Domain-Specific Language (DSL)** written specifically for Zero-Knowledge Proofs (ZKPs).

Circom: A Language for Zero-Knowledge Circuits

- **Domain-Specific Language (DSL)** written specifically for Zero-Knowledge Proofs (ZKPs).
- Requires game rules to be expressed using **arithmetic constraints**.

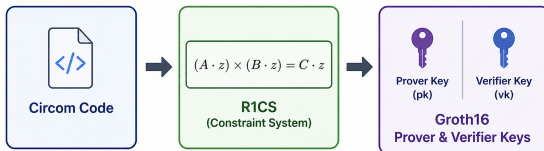
Circom: A Language for Zero-Knowledge Circuits

- **Domain-Specific Language (DSL)** written specifically for Zero-Knowledge Proofs (ZKPs).
- Requires game rules to be expressed using **arithmetic constraints**.
- Uses **Signals** (wires carrying values) and **Components**.

Circom: A Language for Zero-Knowledge Circuits

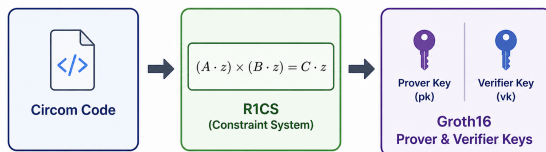
- **Domain-Specific Language (DSL)** written specifically for Zero-Knowledge Proofs (ZKPs).
- Requires game rules to be expressed using **arithmetic constraints**.
- Uses **Signals** (wires carrying values) and **Components**.
- Compiles into a Rank-1 Constraint System (R1CS).

Circom $\rightarrow$ Groth16 Pipeline



(1) Circom (The Compiler)

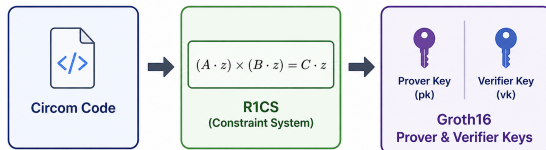
Circom $\rightarrow$ Groth16 Pipeline



(1) Circom (The Compiler)

- Outputs a R1CS.

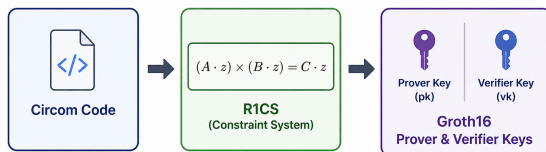
Circom $\rightarrow$ Groth16 Pipeline



(1) Circom (The Compiler)

- Outputs a R1CS.
- Generates a **witness generator**.

Circom $\rightarrow$ Groth16 Pipeline

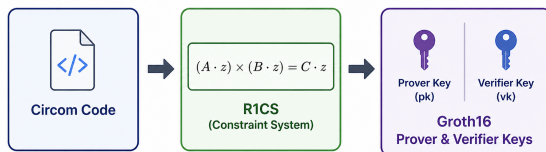


(1) Circom (The Compiler)

- Outputs a R1CS.
- Generates a **witness generator**.

(2) SnarkJS (The Prover & Verifier)

Circom $\rightarrow$ Groth16 Pipeline



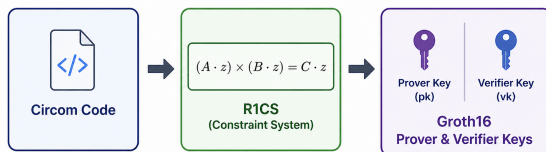
(1) Circom (The Compiler)

- Outputs a R1CS.
- Generates a **witness generator**.

(2) SnarkJS (The Prover & Verifier)

- Takes the R1CS and creates **proving** and **verification** keys.

Circom $\rightarrow$ Groth16 Pipeline



(1) Circom (The Compiler)

- Outputs a R1CS.
- Generates a **witness generator**.

(2) SnarkJS (The Prover & Verifier)

- Takes the R1CS and creates **proving** and **verification keys**.
- Applies the **Groth16** protocol to the witness.

Circuit 1: Board Verification

Circuit 1: Board Verification

- **Private inputs:** Ship (x, y) coordinates and a private salt.

Circuit 1: Board Verification

- **Private inputs:** Ship (x, y) coordinates and a private salt.
- **Public output:** A Poseidon hash (commitment).

Circuit 1: Board Verification

- **Private inputs:** Ship (x, y) coordinates and a private salt.
- **Public output:** A Poseidon hash (commitment).
- Circuit enforces bounds, overlaps, and valid shapes.

Circuit 1: Board Verification

- **Private inputs:** Ship (x, y) coordinates and a private salt.
- **Public output:** A Poseidon hash (commitment).
- Circuit enforces bounds, overlaps, and valid shapes.

```
component eqX[N][N];
component eqY[N][N];

for (var i = 0; i < N; i++) {
  for (var j = i + 1; j < N; j++) {
    eqX[i][j] = IsEqual();
    eqX[i][j].in[0] <== privShipX[i];
    eqX[i][j].in[1] <== privShipX[j];

    eqY[i][j] = IsEqual();
    eqY[i][j].in[0] <== privShipY[i];
    eqY[i][j].in[1] <== privShipY[j];

    eqX[i][j].out * eqY[i][j].out == 0;
  }
}
```

Circuit 1: Board Verification

- **Private inputs:** Ship (x, y) coordinates and a private salt.
- **Public output:** A Poseidon hash (commitment).
- Circuit enforces bounds, overlaps, and valid shapes.

```
component eqX[N][N];
component eqY[N][N];

for (var i = 0; i < N; i++) {
  for (var j = i + 1; j < N; i++) {
    eqX[i][j] = IsEqual();
    eqX[i][j].in[0] <== privShipX[i];
    eqX[i][j].in[1] <== privShipX[j];

    eqY[i][j] = IsEqual();
    eqY[i][j].in[0] <== privShipY[i];
    eqY[i][j].in[1] <== privShipY[j];

    eqX[i][j].out * eqY[i][j].out == 0;
  }
}
```

```
component hasher = Poseidon(15);

for (var i = 0; i < N; i++) {
  hasher.inputs[i] <== privShipX[i];
  hasher.inputs[i + N] <== privShipY[i];
}

hasher.inputs[2 * N] <== privSalt;
pubCommitment <== hasher.out;
```

Circuit 2: Proving the Attack

Circuit 2: Proving the Attack

- **Private inputs:** Ship (x, y) coordinates and a private salt (same).

Circuit 2: Proving the Attack

- **Private inputs:** Ship (x, y) coordinates and a private salt (same).
- **Public inputs:** Player's guess, opponent board commitment, reported result $y \in \{\text{hit}, \text{miss}\}$.

Circuit 2: Proving the Attack

- **Private inputs:** Ship (x, y) coordinates and a private salt (same).
- **Public inputs:** Player's guess, opponent board commitment, reported result $y \in \{\text{hit}, \text{miss}\}$.
- The circuit enforces the unchanged opponent board state and that the reported result is accurate.

Circuit 2: Proving the Attack

- **Private inputs:** Ship (x, y) coordinates and a private salt (same).
- **Public inputs:** Player's guess, opponent board commitment, reported result $y \in \{\text{hit}, \text{miss}\}$.
- The circuit enforces the unchanged opponent board state and that the reported result is accurate.

```
component hasher = Poseidon(2 * N + 1);  
  
for (var i = 0; i < N; i++) {  
    hasher.inputs[i] <== privShipX[i];  
    hasher.inputs[i + N] <== privShipY[i];  
}  
  
hasher.inputs[2 * N] <== privSalt;  
pubCommitment == hasher.out;
```

Circuit 2: Proving the Attack

- **Private inputs:** Ship (x, y) coordinates and a private salt (same).
- **Public inputs:** Player's guess, opponent board commitment, reported result $y \in \{\text{hit}, \text{miss}\}$.
- The circuit enforces the unchanged opponent board state and that the reported result is accurate.

```
component hasher = Poseidon(2 * N + 1);  
  
for (var i = 0; i < N; i++) {  
    hasher.inputs[i] <== privShipX[i];  
    hasher.inputs[i + N] <== privShipY[i];  
}  
  
hasher.inputs[2 * N] <== privSalt;  
pubCommitment === hasher.out;
```

```
signal hitAccumulator[N + 1];  
hitAccumulator[0] <== 0;  
  
for (var i = 0; i < N; i++) {  
    eqX[i] = IsEqual();  
    eqX[i].in[0] <== pubGuessX;  
    eqX[i].in[1] <== privShipX[i];  
  
    eqY[i] = IsEqual();  
    eqY[i].in[0] <== pubGuessY;  
    eqY[i].in[1] <== privShipY[i];  
  
    hitMatch[i] <== eqX[i].out * eqY[i].out  
    hitAccumulator[i + 1] <== hitAccumulator[i]  
    ↪ + hitMatch[i];  
}  
  
pubReportedHit === hitAccumulator[N];
```

- **Poseidon capacity:** The Circom implementation is optimized for a maximum of 16 inputs.

- **Poseidon capacity:** The Circom implementation is optimized for a maximum of 16 inputs.
- **Coordinate cap:** Each ship square needs a coordinate (x, y) . Including the salt, we are therefore capped at 7 squares:

$$\underbrace{2 \cdot 7}_{(x,y) \text{ coordinates}} + \underbrace{1}_{\text{salt}} \leq 16$$

Limitation

- **Poseidon capacity:** The Circom implementation is optimized for a maximum of 16 inputs.
- **Coordinate cap:** Each ship square needs a coordinate (x, y) . Including the salt, we are therefore capped at 7 squares:

$$\underbrace{2 \cdot 7}_{(x,y) \text{ coordinates}} + \underbrace{1}_{\text{salt}} \leq 16$$

- **Conflict:** Standard Battleship requires 5 ships totaling 17 squares.

Limitation

- **Poseidon capacity:** The Circom implementation is optimized for a maximum of 16 inputs.
- **Coordinate cap:** Each ship square needs a coordinate (x, y) . Including the salt, we are therefore capped at 7 squares:

$$\underbrace{2 \cdot 7}_{(x,y) \text{ coordinates}} + \underbrace{1}_{\text{salt}} \leq 16$$

- **Conflict:** Standard Battleship requires 5 ships totaling 17 squares.
- **The fix?**

Outline

- 1 Motivation
- 2 Demo
- 3 Overview of Zero-Knowledge Proofs
- 4 Battleship Circuits
- 5 Merkle Trees**

Naïve Commitment

- Let the hidden board state be

$$x = (x_1, \dots, x_N),$$

where each $x_j \in \{0, 1\}$ indicates whether square j contains a ship.

Naïve Commitment

- Let the hidden board state be

$$x = (x_1, \dots, x_N),$$

where each $x_j \in \{0, 1\}$ indicates whether square j contains a ship.

- At setup, the defender publishes a commitment to the whole board:

$$C = H(x_1, \dots, x_N).$$

Naïve Commitment

- Let the hidden board state be

$$x = (x_1, \dots, x_N),$$

where each $x_j \in \{0, 1\}$ indicates whether square j contains a ship.

- At setup, the defender publishes a commitment to the whole board:

$$C = H(x_1, \dots, x_N).$$

- For a shot at square i , the defender reports

$$y \in \{\text{hit}, \text{miss}\}.$$

Naïve Commitment

- Let the hidden board state be

$$x = (x_1, \dots, x_N),$$

where each $x_j \in \{0, 1\}$ indicates whether square j contains a ship.

- At setup, the defender publishes a commitment to the whole board:

$$C = H(x_1, \dots, x_N).$$

- For a shot at square i , the defender reports

$$y \in \{\text{hit}, \text{miss}\}.$$

- The defender proves

$$\begin{aligned} & \exists x_1, \dots, x_N \\ \text{s.t. } & C = H(x_1, \dots, x_N), \\ & y = f(x_i). \end{aligned}$$

Naïve Commitment: Cost

- **Problem:** Even though we only need to verify the local value x_i , the circuit must re-hash the entire board.

Naïve Commitment: Cost

- **Problem:** Even though we only need to verify the local value x_i , the circuit must re-hash the entire board.
- Groth16 proving cost depends on the number of circuit constraints.

Naïve Commitment: Cost

- **Problem:** Even though we only need to verify the local value x_i , the circuit must re-hash the entire board.
- Groth16 proving cost depends on the number of circuit constraints.
- Re-hashing the full board creates constraints that scale with the size of the game state:

$$\mathcal{O}(N).$$

Merkle Tree Commitment

- **Goal:** Commit to many values using one root hash.

Merkle Tree Commitment

- **Goal:** Commit to many values using one root hash.
- In our case, the values are the board squares $x_1, \dots, x_N$.

Merkle Tree Commitment

- **Goal:** Commit to many values using one root hash.
- In our case, the values are the board squares $x_1, \dots, x_N$.
- The defender publishes the Merkle root

$$R = \text{MerkleRoot}(\ell_1, \dots, \ell_N),$$

where each leaf

$$\ell_j = \text{H}_{\text{leaf}}(j, x_j, r_j)$$

is a commitment to square j .

Merkle Tree Commitment

- **Goal:** Commit to many values using one root hash.
- In our case, the values are the board squares $x_1, \dots, x_N$.
- The defender publishes the Merkle root

$$R = \text{MerkleRoot}(\ell_1, \dots, \ell_N),$$

where each leaf

$$\ell_j = \text{H}_{\text{leaf}}(j, x_j, r_j)$$

is a commitment to square j .

- Here:

x_j = value of square j ,

r_j = private randomness.

Merkle Tree Commitment

- **Goal:** Commit to many values using one root hash.
- In our case, the values are the board squares $x_1, \dots, x_N$.
- The defender publishes the Merkle root

$$R = \text{MerkleRoot}(\ell_1, \dots, \ell_N),$$

where each leaf

$$\ell_j = \text{H}_{\text{leaf}}(j, x_j, r_j)$$

is a commitment to square j .

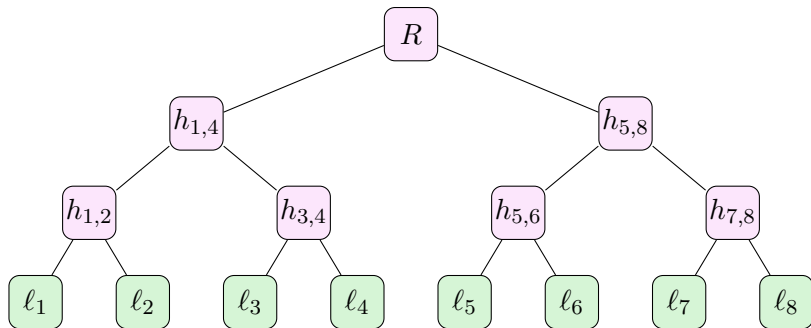
- Here:

x_j = value of square j ,

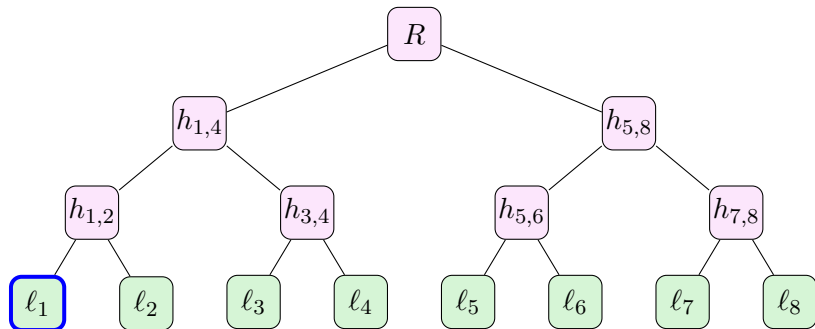
r_j = private randomness.

- R commits to the entire board, but a square can be verified using only its **Merkle path**.

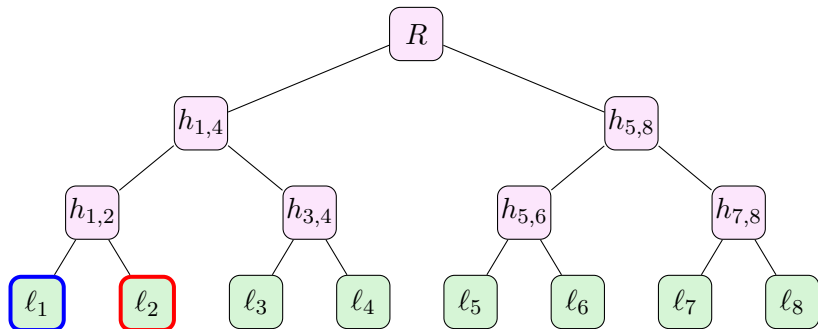
Merkle Tree



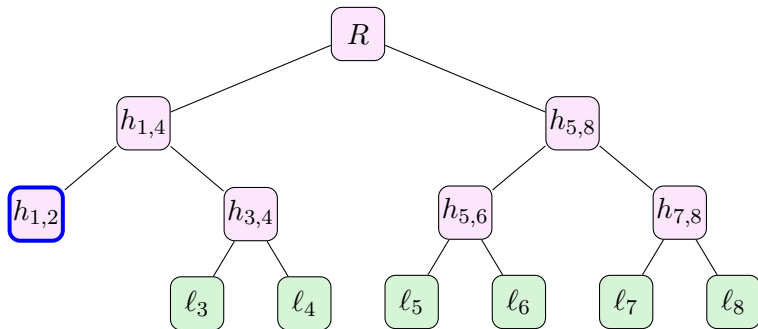
Merkle Tree



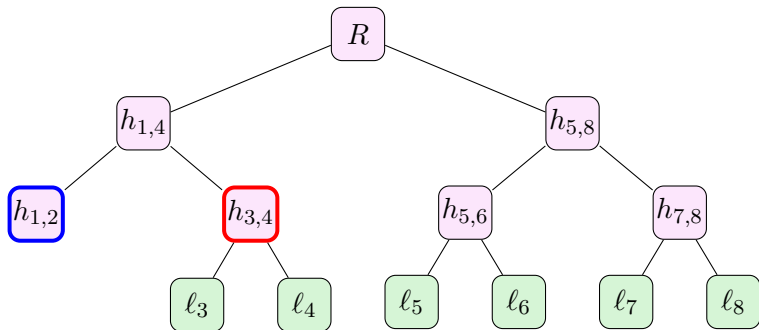
Merkle Tree



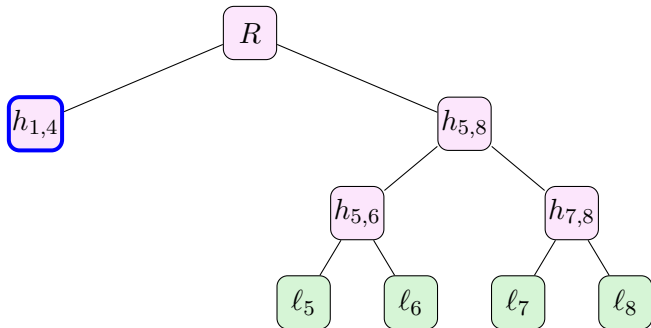
Merkle Tree



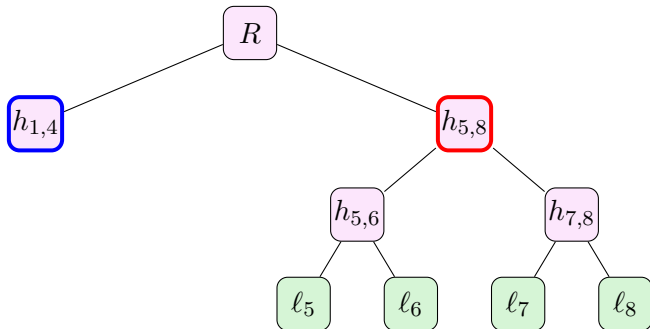
Merkle Tree



Merkle Tree



Merkle Tree



Merkle Tree



Why This Helps

	Naïve	Merkle
Private inputs	$x_1, \dots, x_N$	x_i, r_i, Π_i
Commitment	C	R
Constraints	$\mathcal{O}(N)$	$\mathcal{O}(\log N)$

Thank you!

Questions?